Introduction to Graph Theory

Written by Giancarlo Massimo

Topic Summaries

Topic 1: What is a Graph?

Key Concepts

- Identify a graph and its parts.
- Identify real-world situations where a graph can be used to model a problem.

Key Terms:

- Graph
- Node (Vertex)
- Edge

Topic 2: Types of Graphs

Key Concepts

- Define different kinds of graphs and their properties.
- Classify a graph given a visual representation of it.
- Understand that a tree is a type of graph.
- Understand that a graph can have zero edges

Key Terms:

Types of Graphs	Properties of Graphs
• Directed/Undirected Graph • Cyclic/Acyclic Graph • Regular Graph • Complete Graph • Connected/Disconnected Graph • Weighted/Unweighted Graphs • Planar/Non-Planar Graphs	• Degree (in and out) • Unidirectional/Bidirectional Edge • Adjacency (neighbors) • Path • Cycle • Loop • Connected Components

Topic 3: Graph Representations

Key Concepts

- Identify different representations of graphs.
- Know how to convert between a visual representation of a graph and a tabular representation of a graph.
- Understand pros and cons of different graph representations.

Key Terms:

- Edge List
- Adjacency Matrix
- Adjacency List

Topic 4: Graph Coloring

Key Concepts

- Define a properly colored graph and how it can help solve different problems.
- Explain a bipartite graph in terms of its coloring.
- Explain how the Four Color Theorem relates to planar graphs.

Key Terms:

- Chromatic Number
- Bipartite Graph
- Four Color Theorem

Topic 5: Implementing a Graph

Key Concepts

- Implement a graph in Java according to the needs of the program.

Topic 6: Algorithms

Key Concepts

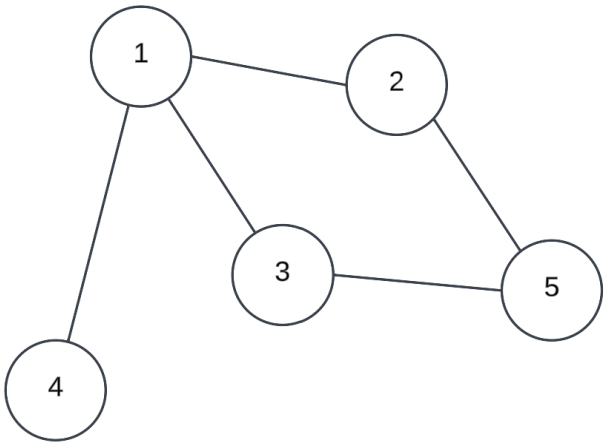
- Understand how to traverse a graph using BFS and DFS.
- Implement a graph traversal in Java using both recursion and iteration.
- Understand the sequence of Dijkstra's Shortest Path Algorithm.
- Know the time complexity (Big O) of each algorithm covered.

Key Terms:

- Breadth-First Search (BFS) and Depth-First Search (DFS)
- Dijkstra's Shortest Path Algorithm

What is a Graph?

A graph is a data structure made up of **nodes** (sometimes called vertices) connected by lines called **edges**.



You can think of a graph like a network, with the nodes containing data and the edges signifying some kind of relationship between that data.

Graph theory is a broad subfield and is important to both computer science and discrete mathematics. The goal of this unit is to provide a foundational understanding of graphs, and it is far from exhaustive. We want to begin to notice graphs in problems that come up often in CS and have another tool at our disposal when programming.

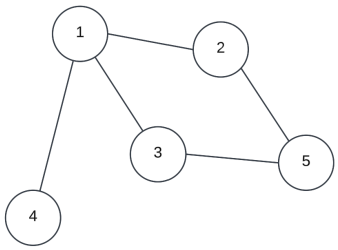
In the real world, we come across graphs nearly every day. They're often a very natural way to model certain situations, including:

- A board game with cities connected by pathways.
- A social network, where users are connected to their friends.
- A transportation system, where points on a map are connected by roads.
- The links between different behaviors of a video game AI.
- The migration patterns of different species.

However, graphs can also appear in more subtle and surprising ways. We'll look at some of those cases here and how framing a problem in terms of a graph can be helpful towards finding a solution.

Types of Graphs

Key Terms



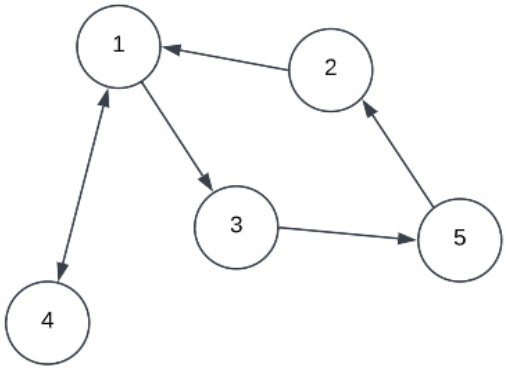
Adjacency - If two nodes are connected by a single edge, they are adjacent. If two edges share a common node, they are adjacent. Adjacent nodes are **neighbors**.

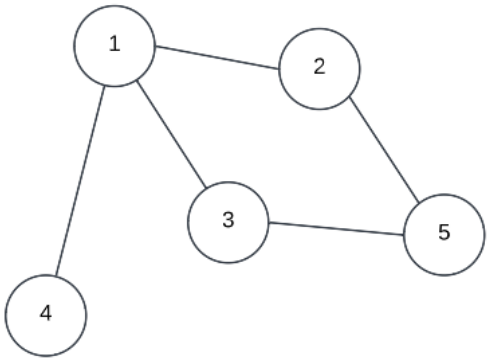
Example: Nodes 1 and 4 are adjacent, and the edge between 1 and 4 is adjacent to the edge between 1 and 3.

Path - A path is a list of adjacent nodes. *Example:* $[1, 3, 5, 2]$, which has a length of 3. Note that the length of a path is the number of *edges* between the start and end nodes.

Directionality and Cycles

Graphs can either be **directed** or **undirected**.

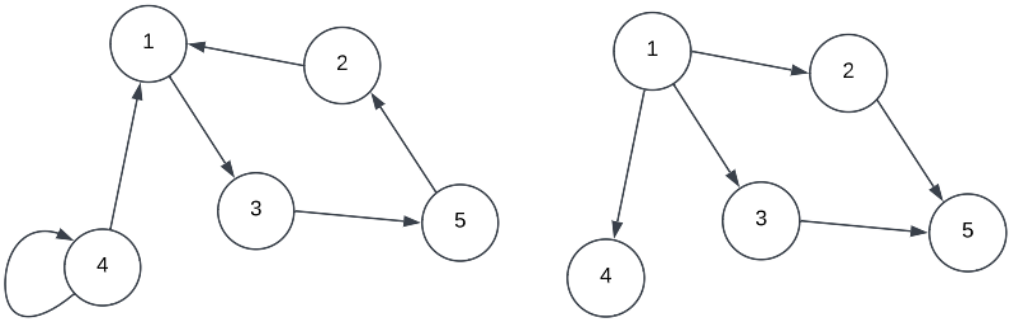




In a directed graph, nodes are only connected in a specified direction, indicated above by arrows at the ends of each edge. It is possible, for example, to move from 1 to 3, but not from 5 to 3. Edges can be **unidirectional** or **bidirectional**, as is the case between 1 and 4.

There is no such specification in an undirected graph. We are free to move in both directions along any edge.

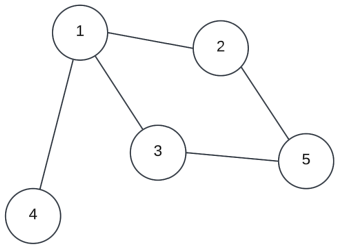
When a directed graph contains a **cycle**, it is called a **cyclic graph**. A cycle is a path that begins and ends at the same node.



Left: This graph contains the cycle $[1, 3, 5, 2, 1]$. Graphs can also have **loops**, as shown by Node 4. *Remember to always include the first and last node in the cycle.*

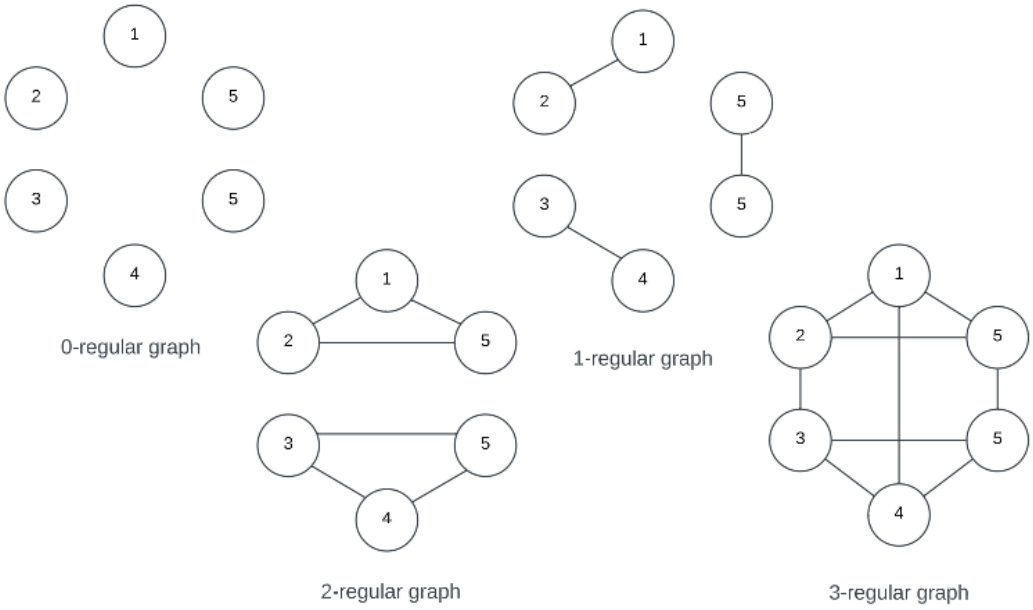
Right: This graph contains no cycles, so it is referred to as a **directed acyclic graph**.

Graphs Defined by Degree



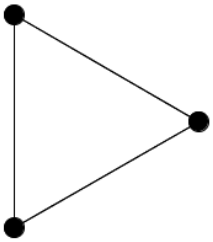
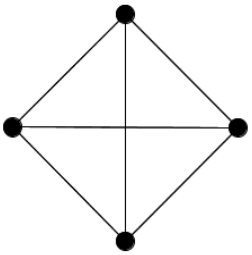
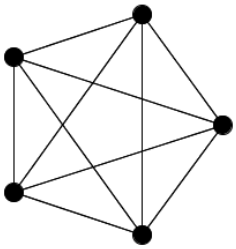
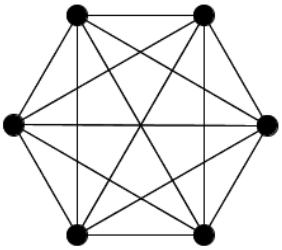
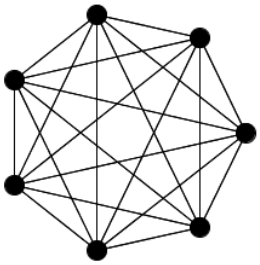
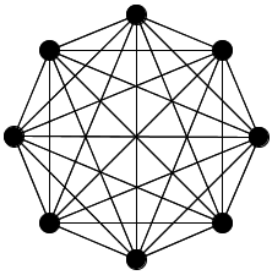
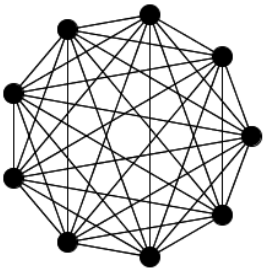
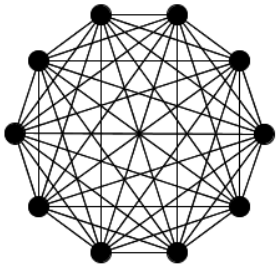
Degree - The degree of a node is the number of edges connected to it. *Example:* Node 1 has a degree of 3. Nodes in directed graphs have an **indegree** and **outdegree**.

A graph is **regular** if all its nodes have the same degree.



b

Likewise, a regular graph is considered **complete** if every node is connected to every other edge. A complete graph with n nodes will contain $n(n - 1) / 2$ edges.

2 Nodes | 1 Edge	3 Nodes | 3 Edges	4 Nodes | 6 Edges
		
5 Nodes | 10 Edges	6 Nodes | 15 Edges	7 Nodes | 21 Edges
		
8 Nodes | 28 Edges	9 Nodes | 36 Edges	10 Nodes | 45 Edges
		

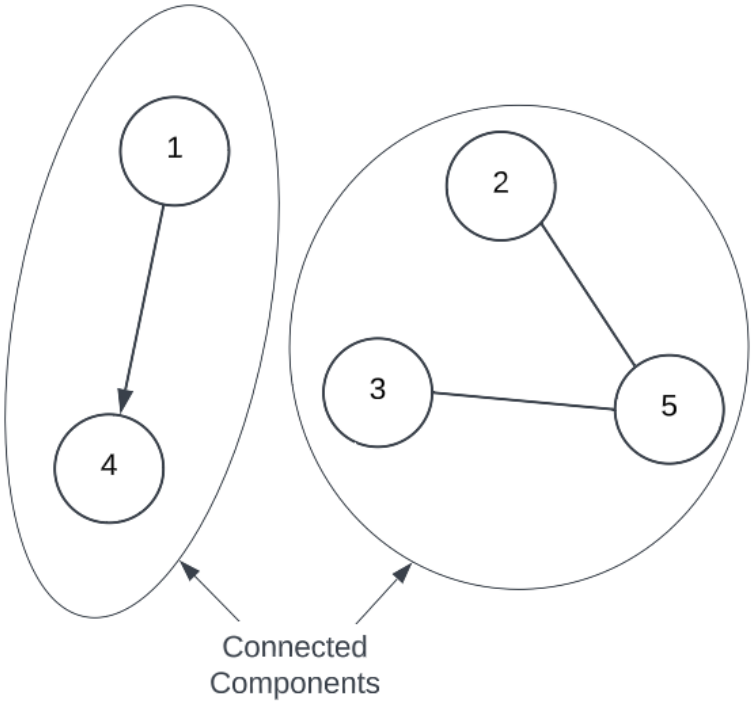
Graphs created using Processing: https://processing.org/

Note: By now you may have noticed that a binary search tree (or any tree, for that matter) looks a lot like a graph. A tree is, in fact, a type of graph. A graph is a tree if it meets the following criteria:

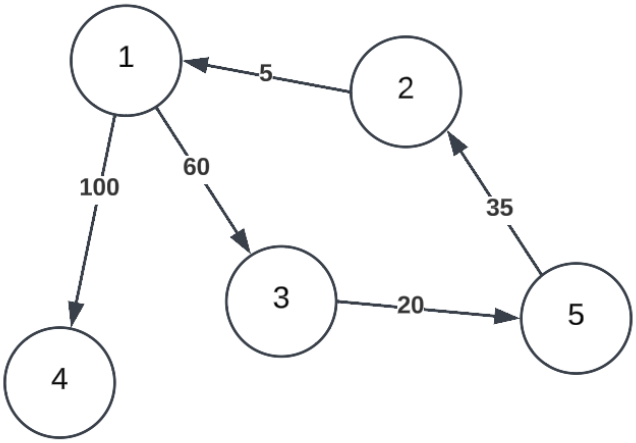
1) It is undirected and contains no cycles.
2) In a tree with n nodes, the number of edges is $n - 1$.
3) There is exactly one path between any two nodes.

Other Types of Graphs

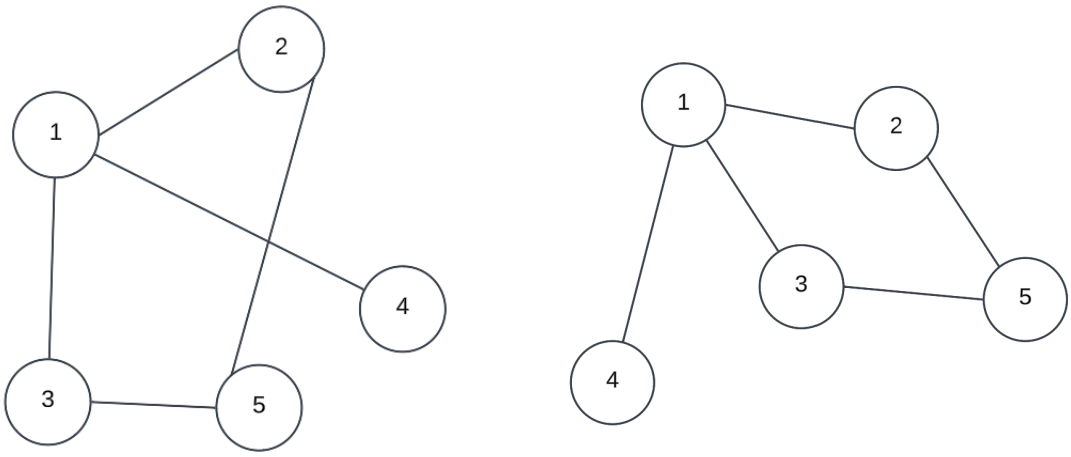
Graphs can either be **connected** or **disconnected**. Disconnected graphs are made up of multiple **connected components**. Note that **not all graphs need to have edges. It is possible for a node to be a connected component by itself.**



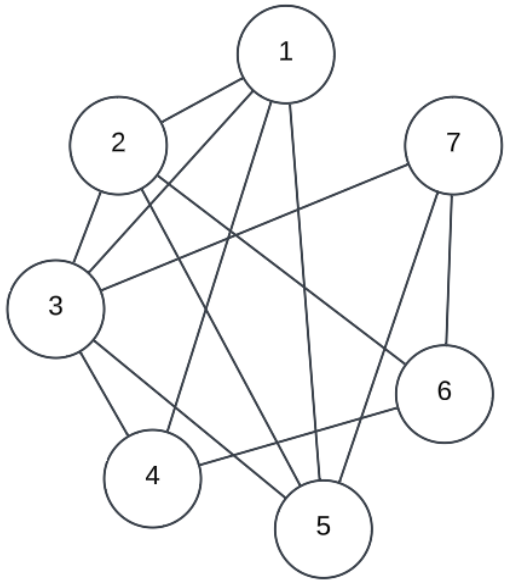
Graphs can also have edges that are **weighted**. You can think of a weighted edge as the “cost” to move across it. For example, it might take very long to move across a highway in a traffic jam but very little to drive across a less busy road.



Finally, a **planar graph** is a graph whose edges never intersect. Note that just because a graph *is* drawn with intersecting edges doesn't mean that it *must* be drawn with intersecting edges. The following two graphs are both planar (and both exactly the same graph):



This graph, however, is **non-planar** because it is impossible to draw without any edges intersecting each other:



Planar graphs are especially useful for representing situations where intersections are impossible or undesirable, such as a railroad network or canal system. They also have interesting properties that we will explore further in *Topic 4: Graph Coloring*.

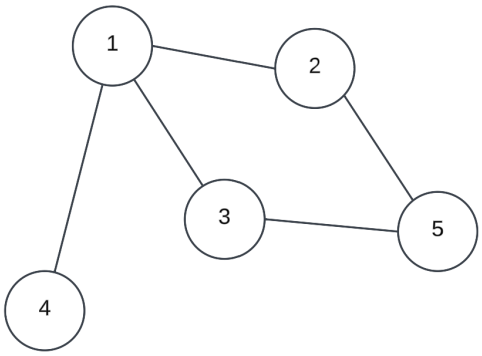
Graph Representations

Up until now, we've been representing graphs visually, with circles showing nodes and lines showing edges. This format is convenient for us to look at, but there are also **tabular representations** of graphs that are more suited to storing them on a computer. We'll go over three such representations and weigh their pros and cons.[1]

Edge List

One of the most simple ways to represent a graph is with an **edge list**. For each set in the edge list, $\{a,\ b\}$, there is an edge between a and b. We can either specify the graph to be undirected—in which case $\{a,\ b\}$ would go in both directions—or directed—in which case $\{a,\ b\}$ would only move from a to b and $\{b,\ a\}$ would denote the other direction. We can account for weighted edges by adding a third number, $\{a,\ b,\ w\}$, to the set.

$$[\,\{1,\ 4\},\ \{1,\ 3\},\ \{1,\ 2\},\ \{2,\ 5\},\ \{3,\ 5\}\,]$$



Edge lists are useful for constructing a graph based on input because they can easily be represented in a text file as shown on the right.

Nevertheless, by themselves they are limited because while it's easy to iterate through each edge, there is no clear way to iterate through nodes.

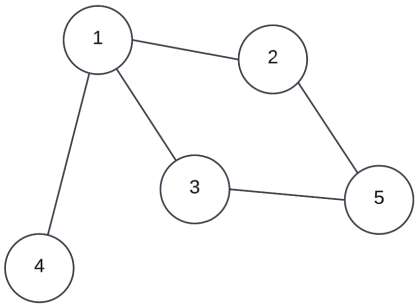
```
     Number of edges → 5
Each line is an edge → 1 4
                       1 3
                       1 2
                       2 5
                       3 5
```

[1] You'll notice that we haven't yet begun to store very useful data in graphs. The purpose of this topic is to introduce you to these abstract representations so that later in *Topic 5*, we can expand on them and make them more flexible using object-oriented programming.

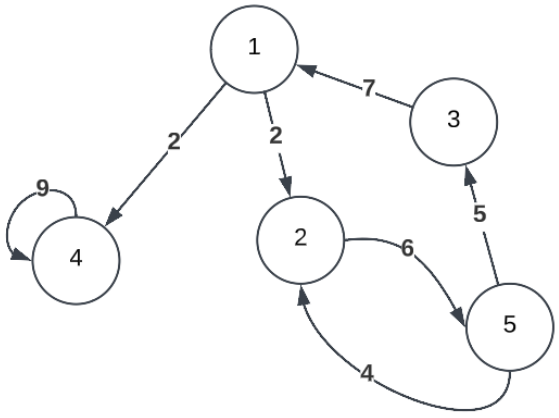
Adjacency Matrix

An **adjacency matrix** is a square, V by V 2D array where V is the number of vertices (nodes) in the graph. An edge exists from a to b if $mat[a][b] \geq 1$. For unweighted graphs, $mat[a][b]$ is binary—either 0 or 1—while weighted graphs allow for larger values of $mat[a][b]$. If a graph is undirected, it is always true that $mat[a][b] = mat[b][a]$. If a graph has no loops, it is always true that $mat[a][a] = 0$.

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} \end{array}$$



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 2 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 9 & 0 \\ 0 & 4 & 5 & 0 & 0 \end{bmatrix} \end{array}$$



Advantages

- Since we are using a matrix, we can remove an edge in $O(1)$ time.
- Likewise, checking whether there is an edge between two nodes takes $O(1)$.

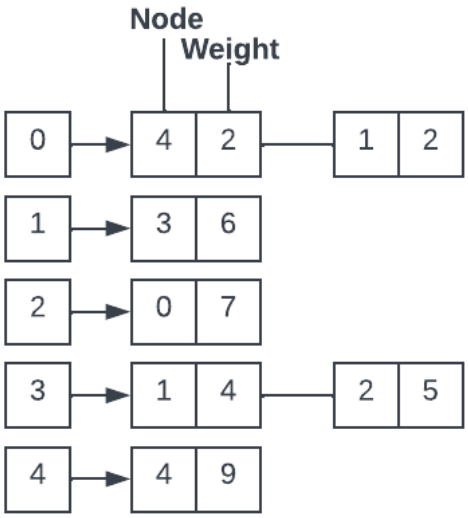
Disadvantages

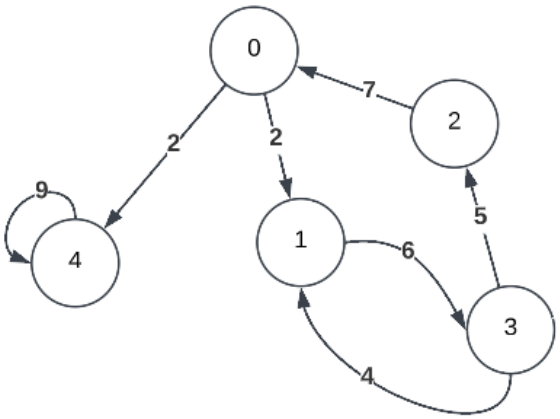
- Since we are using a 2D array, the we need $O(V^2)$ space.
- Adding a node takes $O(V^2)$ time.
- Finding a node's adjacent nodes takes $O(V)$ time—higher than with other representations.

Adjacency List

An **adjacency list** is an array of lists. The length of the array is the number of nodes, and each entry in the array represents a node. Each list contains that node's adjacent nodes. In an undirected graph, it is always true that if $adj[a]$ contains b then $adj[b]$ contains a. For weighted graphs, we can use an array of lists of pairs. For example: $adj[a] = [\{b, w_1\}, \{c, w_2\}, \{d, w_3\}, \ldots]$

An adjacency list can also be implemented using a Map (and in ways that are more flexible to us), which we will see in *Topic 5: Implementing a graph.*





Advantages

- Since we're using a 1D array, they only use $O(V + E)$ space.
- Finding adjacent nodes only requires iterating through that node's adjacencies.

Disadvantages

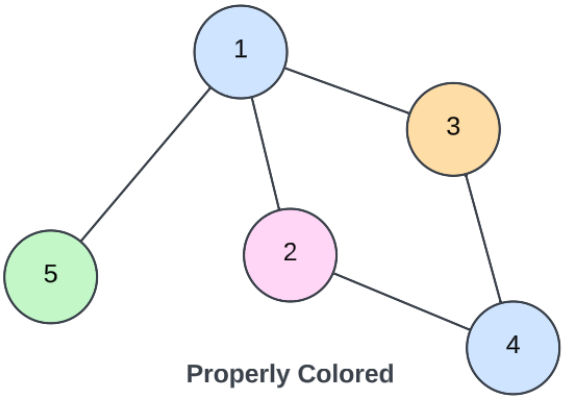
- Checking whether there's a path between one node and another takes $O(V)$ time.
- Not much—adjacency lists are *generally* the best overall option.

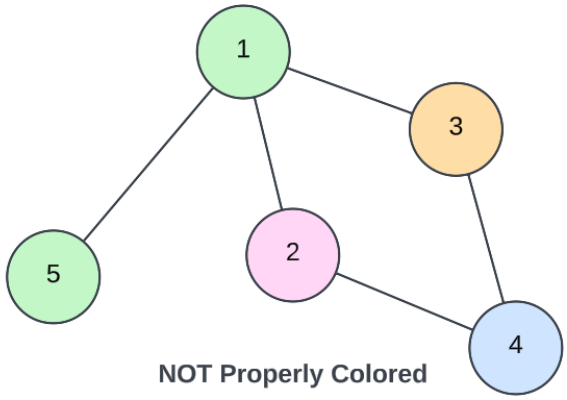
Adjacency lists are popular because they're efficient at storing **sparse** graphs. In many real-world situations, graphs are sparse, meaning that they contain significantly fewer edges than there would be in a complete graph of N nodes (in other words, $E \ll N^2$).

For example, Facebook has almost 3 billion users, but each user on average only has about 338 friends. If each pair of friends is an edge, then the total number of edges in a graph of Facebook users would be far less than the total number of possible edges in a 3-billion-node graph.

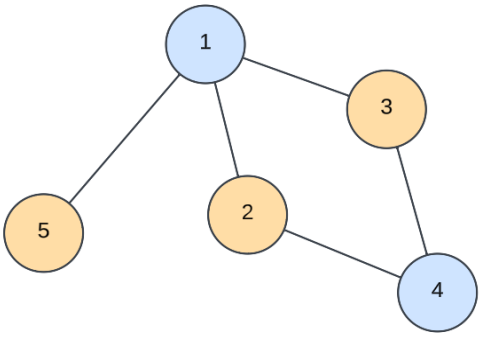
Graph Coloring

Now we're going to introduce the concept of a colored graph: a graph is considered **properly colored** if no two adjacent nodes have the same color.[2] [3]





The **chromatic number** of a graph G is the minimum number of colors needed to properly color it, notated as $\chi(G)$. A graph G is also referred to as N-colorable if it can be colored with N or fewer colors—in other words, if $\chi(G) \leq N$. The chromatic number of the above graph is 2, because it can be colored with 2 colors.

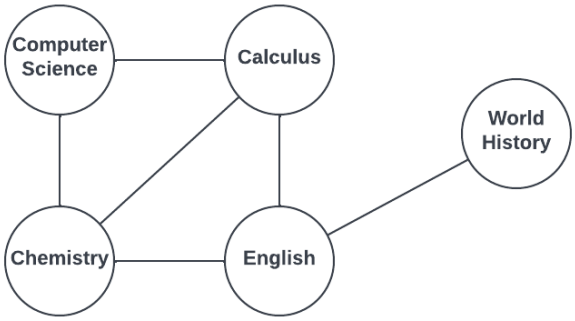


Using Colored Graphs for Scheduling

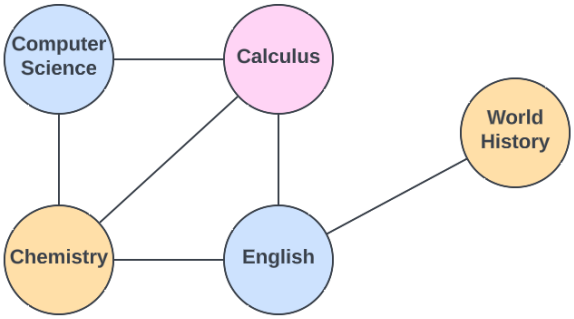
Now we turn our attention to a common use for colored graphs to study how they can appear in seemingly unexpected places. Suppose a university needs to schedule final exams. Certain exams can't be scheduled at the same time because their classes have overlapping students. The university would still like to schedule exams in the fewest number of days possible, though.

[2] Note that a graph that contains a loop can never be properly colored.
[3] Not all colored graphs will follow this rule (e.g. Red-Black Trees), but this is the most common rule used for graph coloring, and it is useful in many situations. Unless otherwise specified, "properly colored graph" refers to this definition.

Here's how we can use graph theory to schedule exams for the university. Let's make a graph with nodes representing each class. An edge between two classes means they have overlapping students and their exams cannot be scheduled on the same day.



Now suppose we have a maximum of five days to schedule exams. Let's assign each day a color: C_1 = blue, C_2 = yellow, C_3 = pink, C_4 = green, C_5 = red. Now let's properly color the graph using the fewest colors possible.



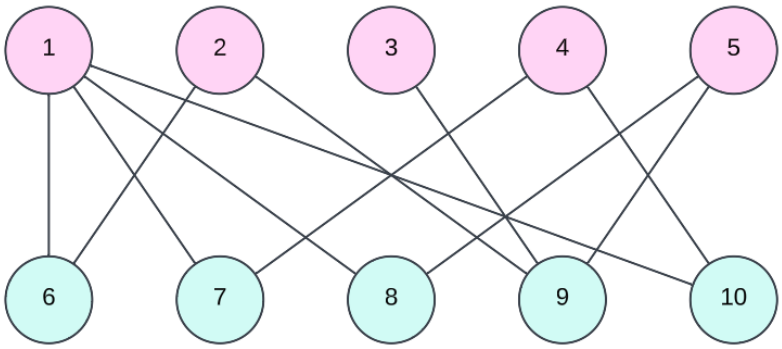
The exact coloring can vary, but regardless of how we color it, we should arrive at a chromatic number of 3. We can schedule Computer Science and English on Day 1, Chemistry and World History on Day 2, and Calculus on Day 3. We finish all our exams in three days with no conflicts. While this example is obviously oversimplified, this kind of analysis can be crucial in a more realistic scenario.[4]

Note: There is currently no known efficient algorithm to find the chromatic number of a graph (the problem is considered NP-Hard). There are, however, a great number of algorithms and heuristics that guarantee an output within a certain range of the actual chromatic number. Even simple algorithms can be used to produce fairly efficient colorings that are practically useful. Can you come up with an algorithm to color a graph as few colors as possible?

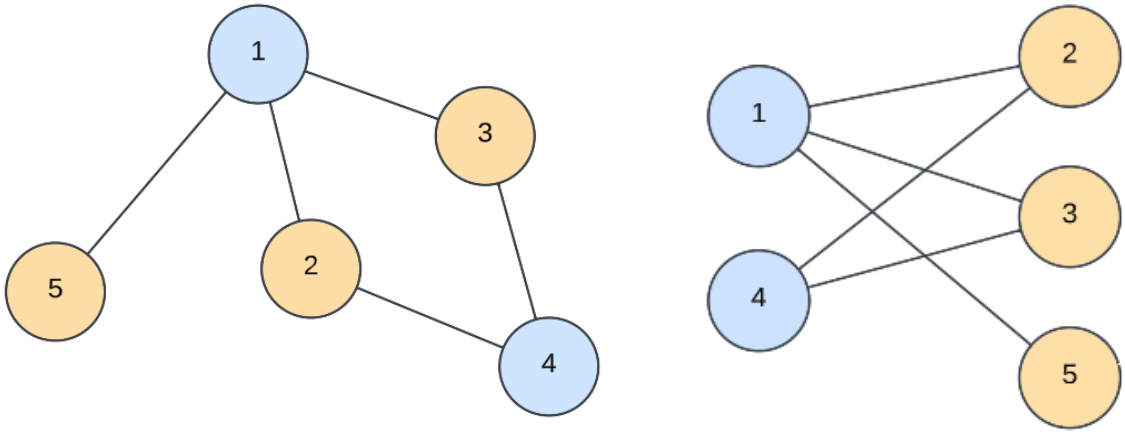
[4] For a deeper look into this topic, see MIT OpenCourseWare, *Lec 6 | MIT 6.042J Mathematics for Computer Science, Fall 2010*

Bipartite Graphs

With our knowledge of graph coloring, we can now introduce a new kind of graph. A graph is **bipartite** if and only if it is 2-colorable:

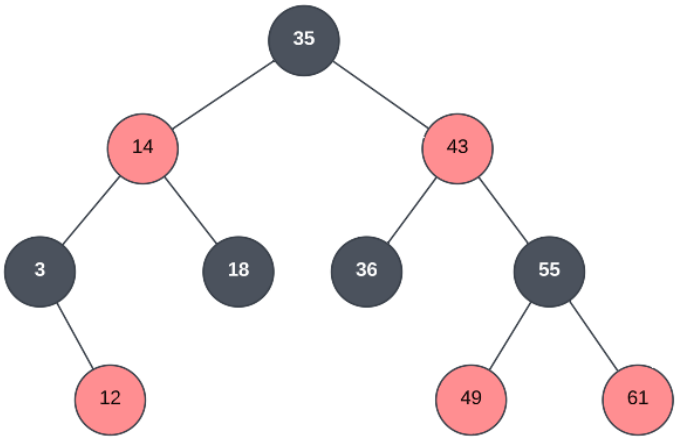


As you can see, we can divide a bipartite graph into two distinct sets, where there is never an edge between any of the nodes inside each of the sets—only across sets. Note, however, that just because a graph doesn't *look* bipartite doesn't mean it *isn't*. Visually, the most reliable way of telling whether a graph is bipartite or not is to see if it's 2-colorable. In fact, if we look at the graph we've been using throughout these notes and rearrange it—it's bipartite.

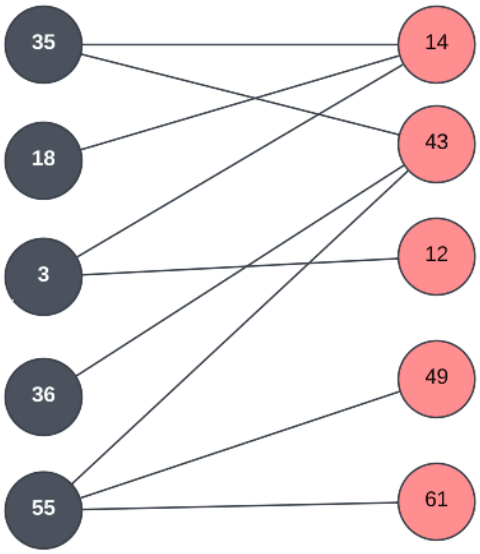


Nevertheless, there are other properties that can help us verify a bipartite graph. Namely, a bipartite graph will contain no odd-length cycles (you can verify this in the graph above).[5] They can even contain no cycles at all, and by now, you may recall another 2-colorable data structure that (by the definition of a tree), contains no cycles.

[5] Recall from the definition of a path (and by extension, a cycle) that the length of a cycle is measured by the number of *edges* between the start and end nodes, not by the number of nodes in the path.



In fact, every tree is bipartite.



Bipartite graphs are useful in real life for several reasons which we cannot detail in-depth here—anywhere from cancer research to matching problems such as the Stable Marriage Problem. In 2009, Netflix offered a $1 million prize to anyone who could improve their recommendation algorithm by 10%. The winning algorithm, "BellKor's Pragmatic Chaos," (and many others) made use of bipartite graphs, and so do many modern machine-learning neural networks.

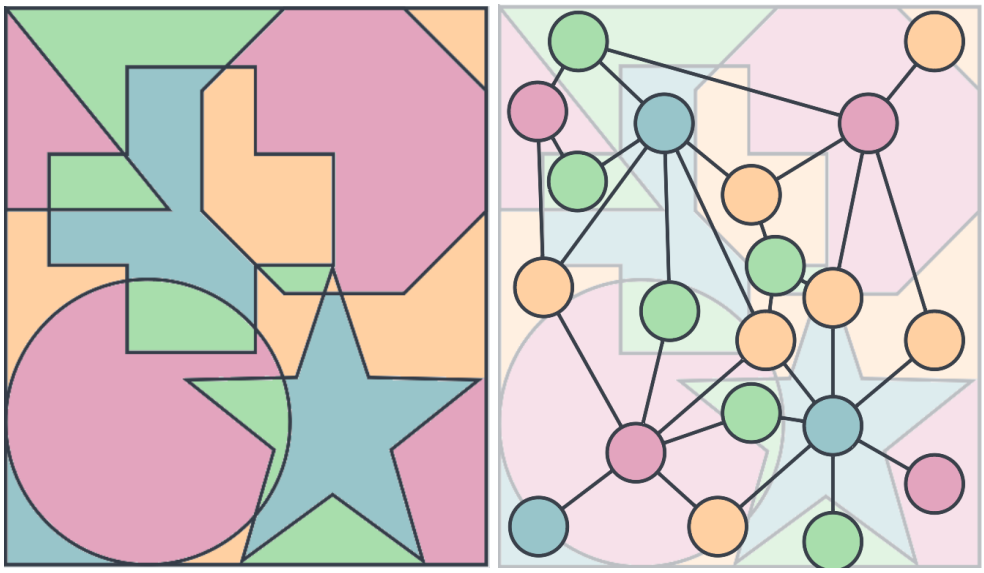
Four Color Theorem

Suppose you're in charge of making a map of the United States. We want each state to be colored in, but we don't want bordering states to have the same color. How many colors would you need? It turns out the answer is four. What about Europe? Also four. Asia? Four. The world? Still four.[6]

The **four color theorem** states that any map can be colored in such a way. But more importantly for us, any map of different regions can be turned into a graph—each region is a node, and region a borders region b if an edge exists between a and b. This will always result in a planar graph.

Therefore, we can put the four color theorem in terms of a graph: any planar graph G with no loops has a chromatic number $\chi(G) \leq 4$.

[6] You can find a good examples here: https://geology.com/world/world-map.shtml



The four color theorem remained unproven for over a century after it was first proposed in 1852. In 1976, it became one of the first theorems to be proved using a computer.

Implementing a Graph

Now that we have a base-level understanding of graph theory, let's look at how we might implement a graph in Java. You can try a literal implementation of the graph representations we covered in *Topic 3*, but we have other options, too. We'll go over some ways you might approach implementing a graph, with the goal of having the flexibility to meet the needs of different programs.

The first option is to simply use a map. Either a `HashMap` or `TreeMap` will work, depending on whether you want nodes to be sorted or not.

```
HashMap<Integer, TreeSet<Integer>> graph = new HashMap<>();

graph.put(1, new TreeSet<>(Arrays.asList(2, 3, 5)));
graph.put(2, new TreeSet<>(Arrays.asList(1, 4)));
graph.put(3, new TreeSet<>(Arrays.asList(1, 4)));
graph.put(4, new TreeSet<>(Arrays.asList(2, 3)));
graph.put(5, new TreeSet<>(Arrays.asList(1)));
```

Sometimes though, nodes need to store more than just a single data type. We can create a `Node` class to have complete control over the data we hold.

```
HashMap<Node, TreeSet<Node>> graph = new HashMap<>();
...
class Node {
   int id;
   double value;
   String color;

   ...
}
```

Now with a `Node` class, we might want to keep a list of nodes and let each node keep track of its adjacencies, doing away with a map entirely.

Notice how this implicitly creates an adjacency list.

```
List<Node> nodes = new LinkedList[7]<>();
...
class Node {
   int value;
   List<Node> neighbors;

   ...
}
```

Now suppose that we're making a board game with cities that can be connected by roads. Players can own roads, give them a color, and upgrade them. In cases like these, the edges themselves need to hold data, so we create an `Edge` class.

```
class Node {
   int value;
   List<Edge> edges;

   ...
}

class Edge {
   Player player;
   String color;
   int level;

   Node a, b;

   ...

   public Node getOtherNode(Node node) {
       return node == a ? b : (node == b ? a : null);
   }
}
```

Finally, we might want to encapsulate our graph in its own class, adding whatever functions to it we might find useful. What matters most here is that your implementation works well for the program you're specifically writing. You might use some, none, or a combination of the techniques described here. A data structure is a tool, not a restriction.

[7] Note that the choice here between a `LinkedList` and an `ArrayList` is mainly going to depend on what you use your graph for, as well as memory considerations. As a general rule, `ArrayList` works well for accessing data and `LinkedList` works well for frequently adding and removing data.

```
public abstract class Graph {
   private LinkedList<Node> nodes;
   private LinkedList<Edge> edges;

   public Graph() {
       nodes = new LinkedList<>();
       edges = new LinkedList<>();
   }

   public abstract void addNode(Node node);
   public abstract void addEdge(Edge edge);
   public abstract void addEdge(Node a, Node b, boolean unidirectional);
   public abstract void removeNode(Node node);
   public abstract void removeEdge(Edge edge);

   public abstract boolean connected(Node a, Node b);
   public abstract boolean adjacent(Node a, Node b);
   public abstract boolean distance(Node a, Node b);
}

class Node {
   private int value;
   private List<Edge> edges;

   ...
}

class Edge {
   private int weight;
   private Node a, b;

   ...

   public Node getOtherNode(Node node) {
       return node == a ? b : (node == b ? a : null);
   }
}

class UnidirectionalEdge extends Edge {
   ...

   @Override
   public Node getOtherNode(Node node) {
       return node == a ? b : null;
   }
}
```

Algorithms on a Graph

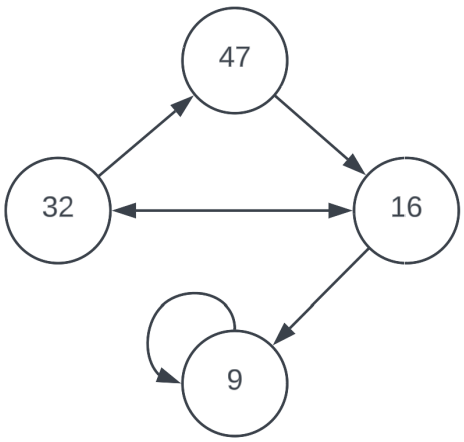
Now that we have a graph implemented in Java ready to use, we'll look at two well-established algorithms for traversing the nodes in a graph and one for finding the shortest distance between two points on a weighted graph. For the following traversal algorithms, we will be using an `ArrayList` of nodes with the following `Node` class:

```
ArrayList<Node> nodes = new ArrayList<Node>();
...
class Node {
	int value;
	int index;
	ArrayList<Node> neighbors;

	public Node(int value, int index) {
		this.value = value;
		this.index = index;
		neighbors = new ArrayList<Node>();
	}

	public void addEdge(Node node) {
		neighbors.add(node);
	}
}
```

And we will be considering the following graph[8]:



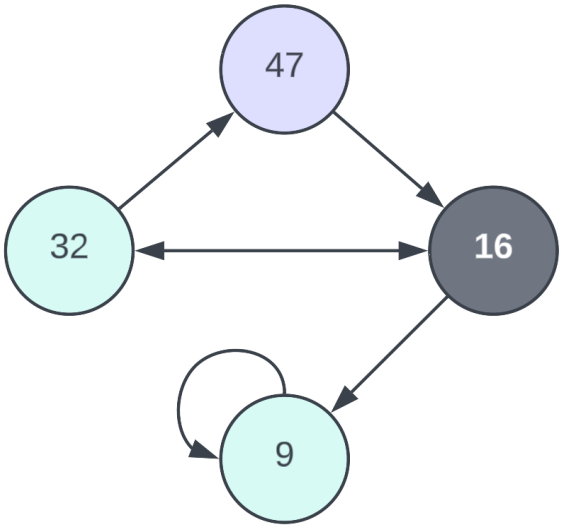
Note that we'll be traversing *connected* graphs. You will need to perform these algorithms for all connected components on a disconnected graph.

[8] Graph modified from Geeks for Geeks: *BFS for Graph*

Breadth First Search

Time Complexity: $O(V + E)$

Breadth First Search involves visiting all nodes equidistant from the starting node before moving to the next level away from the starting node. There are often multiple valid BFS traversals for one graph. For example, starting from Node 16:



16, 32, 9, 47
16, 9, 32, 47

```
public static void BFS(Node starting, ArrayList<Node> nodes) {
	Queue<Node> queue = new LinkedList<Node>();
	queue.add(starting);

	boolean[] visited = new boolean[nodes.size()];
	visited[starting.index] = true;

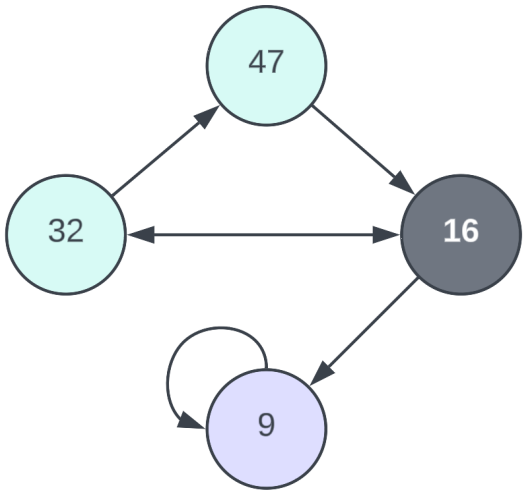
	while (!queue.isEmpty()) {
		Node node = queue.remove();
		visited[node.index] = true;
		System.out.println(node.value);

		for (Node neighbor : node.neighbors) {
			if (!visited[neighbor.index]) {
				queue.add(neighbor);
			}
		}
	}
}
```

Depth First Search

Time Complexity: $O(V + E)$

Depth First Search involves visiting all nodes adjacent to the starting node as deep as possible before following down the next adjacent node. There are often multiple valid DFS traversals for one graph. For example, starting from Node 16:



16, 32, 47, 9
16, 9, 32, 47

```
public static void DFS(Node node, boolean[] visited) {
	visited[node.index] = true;
	System.out.println(node.value);

	for (Node neighbor : node.neighbors) {
		if (!visited[neighbor.index]) {
			DFS(neighbor, visited);
		}
	}
}
```

It is possible to implement both DFS and BFS using both iteration and recursion, but we have chosen iteration for BFS and recursion for DFS because they are generally easier to write. BFS and DFS are usually used to search for a target node. Though both BFS and DFS have the same time complexity, you should choose one over the other depending on which you think will find the desired node first. For example, you should use BFS for problems where you think you will find the target node a short distance away from the start, but DFS if you think it will be far down a particular path.

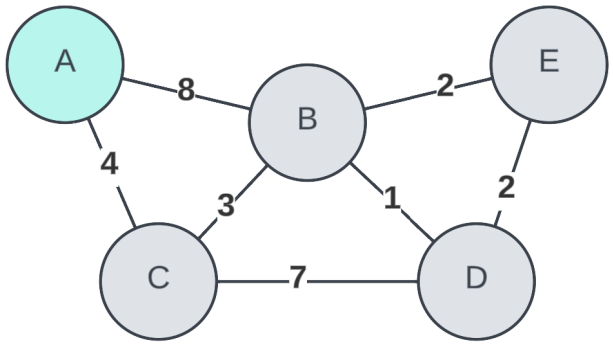
Dijkstra's Algorithm

Time Complexity (when using a Priority Queue): $O(E + V(logV))$

Dijkstra's Shortest Path Algorithm[9] finds the shortest path between two nodes on a weighted graph, where the weights are the "cost" or "time" it takes to move along that edge. This is a very common problem, especially in navigation systems. Note that you will not be required to know how to implement Dijkstra's Algorithm, but you should know the sequence of it. Here's how it goes:

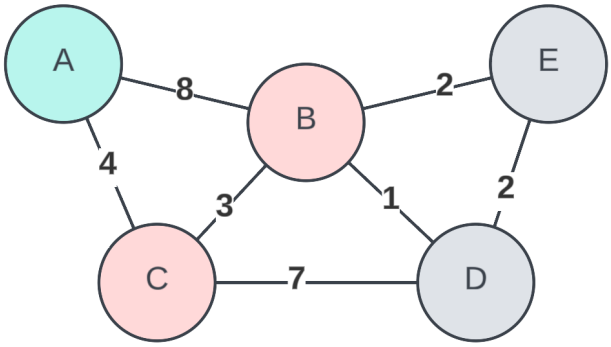
We'll need to keep track of all visited nodes (colored in blue) and the minimum distance discovered to each node (shown by the table).

We'll start by marking the start node as visited. Since we are already at A, the distance to A is zero. Since we haven't explored anywhere else, mark the distance from A to all other nodes to be infinity.



A	0
B	∞
C	∞
D	∞
E	∞

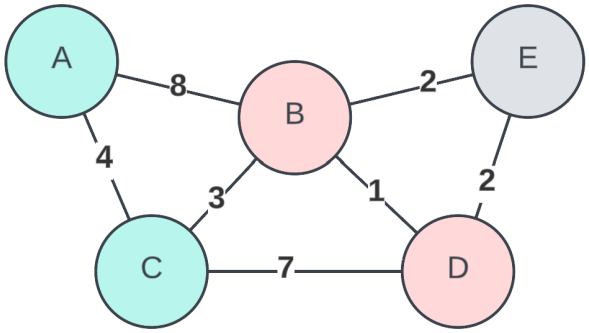
Now we'll check all of A's unvisited neighbors, updating our table with the currently known minimum distance to each node. Note that we sort the table by distance from least to greatest.



A	0
C	4
B	8
D	∞
E	∞

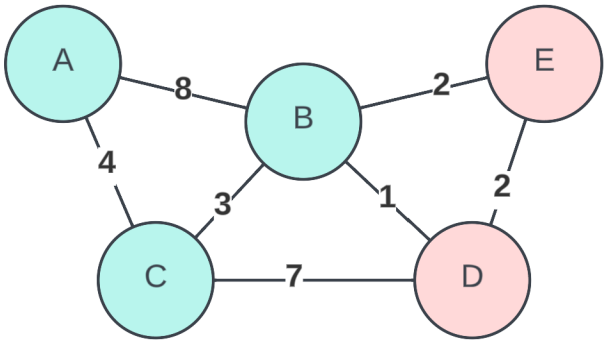
[9] Dijkstra's Algorithm is very well-known, but it's far from the only algorithm used for this purpose. For further reading, see the A* Algorithm (pronounced "a-star").

Of the neighbors nodes we just checked, pick the node with the least distance, C. Mark it visited and repeat the previous steps with C's unvisited neighbors, B and D.



A	0
C	4
B	7
D	11
E	∞

Note that the current shortest path in our table to D goes through $[A,\ C,\ D]$, which is different from the true shortest path of $[A,\ C,\ B,\ D]$. We will see that by the time D is marked visited, we will have found the shortest path to it. E is still unvisited, so its distance remains infinity in our table.



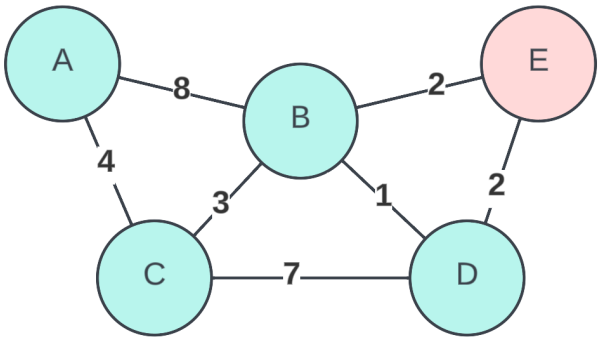
A	0
C	4
B	7
D	8
E	9

We're about to mark D as visited, but by now, you may have noticed that once we mark a node as visited, we have found the shortest path to it. This is because Dijkstra's is a kind of greedy algorithm, meaning that we always make decisions based on the best immediate next step. In Dijkstra's Algorithm, we always explore the unvisited node with the current shortest known distance in our table.

So, when we choose to explore D next, we are doing so because the current shortest distance to it is less than the current shortest distance to E, which is B's other unvisited neighbor. If the shortest path to D instead went through E (imagining for example, that both of E's edges have a weight of zero), then we would have chosen to explore E before D because our table at this step would show:

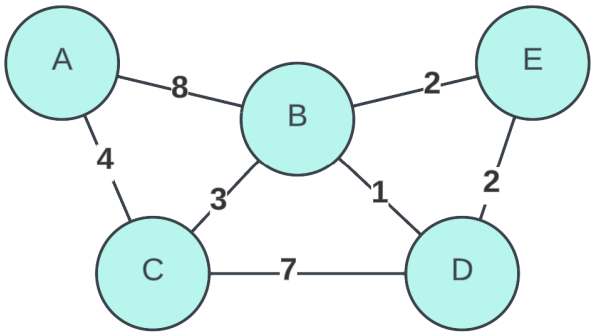
E	7
D	8

Mark D as visited and check its only unvisited neighbor, E. The shortest distance to E going through D is 10, longer than the current shortest path, so nothing changes.



A	0
C	4
B	7
D	8
E	9

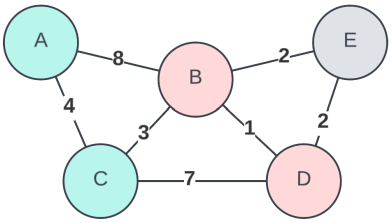
Mark E as visited. We have visited all nodes, and the algorithm is complete. Looking at the table, you can see that we have computed the minimum distance to each node starting from A.



A	0
C	4
B	7
D	8
E	9

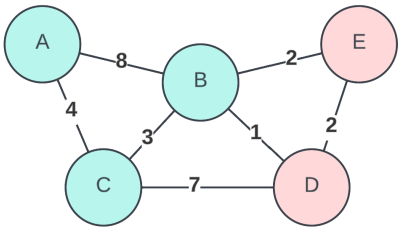
In this case, we're only interested in finding the minimum distance to each node. If we were trying to find the actual path to a node, we would need to modify the algorithm. Each node would need to reference the previous node along the shortest path.

For example, revisit this step: The current minimum distance to D is 11 going through C.



D_C	11

However, in the next step, we find that the shortest distance to D is 8 through B. Do this for all nodes in the table, and you will be able to backtrack to the start along the shortest path.



D_B	8

Resources and Further Reading

Textbook Chapters

- Chapters 6.4 and 12 - Johan Sannemo, Principles of Algorithmic Problem Solving
- Chapters 11 - 13, Antti Laaksonen, Competitive Programmer's Handbook
- Chapter 10 - Darren Yao, An Introduction to the USA Computing Olympiad

Videos

- Introduction to Graph Theory: A Computer Science Perspective
- Dijkstra's Algorithm - Computerphile
- How Dijkstra's Algorithm Works
- Dijkstra's algorithm in 3 minutes

Helpful Websites

- University of San Francisco - Graph Algorithm Visualizations
- Geeks for Geeks - Graph Data Structure And Algorithms
- Geeks for Geeks - Graph and its Representations
- Khan Academy - Representing Graphs
- Geeks for Geeks - BFS for a Graph
- Geeks for Geeks - DFS for a Graph
- Geeks for Geeks - Dijkstra's Shortest Path Algorithm
- Programiz - Dijkstra's Algorithm

University Notes

- UT Austin - Jayadev Misra, Course Notes for CS336: Graph Theory
- UT Austin - Gordon S. Novak Jr., CS 314: Data Structures: Lecture Notes
- Penn State - Christopher Griffin, Graph Theory: Penn State Math 485 Lecture Notes
- MIT OpenCourseWare - 6.042J Chapter 5: Graph Theory, Fall 2010